

plot_summary

July 27, 2026

plot of log mse vs eigenvalues in samples of both close to and far away from the local minimum

```
[1]: import os, sys
import numpy as np
import matplotlib.pyplot as plt
sys.path.insert(0, os.path.abspath(os.path.join(os.getcwd(), '..')))
from SOBOL_scan import *
import pickle

[2]: with open('../results/benchmark_scan_multi/SOBOL_results_1.pkl', 'rb') as f:
    # X is 1,2,3 for different objectives
    results_1, res_close, res_far = pickle.load(f)

log_mse_arr=[]
eigval_1_arr=[]
eigval_2_arr=[]
eigval_3_arr=[]
A_VARS = {
    1: ["I_1", "current", lambda x: x],
    3: ["I_3", "current", lambda x: x],
    10: ["I_10", "current", lambda x: x],
}
A_OBJ = {
    11: [{"measure": ["x", "dispersion"], "goal": 0.0, "weight": 1.0}]
}
for sub_sample in res_close:
    current=sub_sample["current"]
    I_dict={"I_1": current[0], "I_3": current[1], "I_10": current[2]}

    mse=compute_weighted_MSE(I_dict, A_VARS, A_OBJ)[0]
    log_mse_arr.append(np.log10(mse))
    H=compute_hessian_phi(I_dict, A_VARS, A_OBJ)[0]
    eigvals=compute_eig(H)[0]
    # print(eigvals)
    eigvals.sort()
    eigvals = eigvals[::-1]
    eigval_1_arr.append(eigvals[0])
    eigval_2_arr.append(eigvals[1])
```

```

    eigval_3_arr.append(eigvals[2])
plt.scatter(log_mse_arr, eigval_1_arr, label='Eigenvalue 1')
plt.scatter(log_mse_arr, eigval_2_arr, label='Eigenvalue 2')
plt.scatter(log_mse_arr, eigval_3_arr, label='Eigenvalue 3')
plt.xlabel('log(MSE)')
plt.ylabel('Eigenvalues of Hessian')
plt.title('Eigenvalues of Hessian vs log(MSE) for samples close to local_
    ↪minimum')
plt.legend()
plt.show()

```

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_38204\4055894791.py in <cell line: 0>()
    20     mse=compute_weighted_MSE(I_dict, A_VARS, A_OBJ)[0]
    21     log_mse_arr.append(np.log10(mse))
----> 22     H=compute_hessian_phi(I_dict, A_VARS, A_OBJ)[0]
    23     eigvals=compute_eig(H)[0]
    24     # print(eigvals)

c:\Users\yilu2\my_files\research\FELsim\experiment\SOBOL_scan.py in
    ↪wrapper(*args, **kwargs)
    24     def wrapper(*args, **kwargs):
    25         start_time = time.time()
----> 26         result = func(*args, **kwargs)
    27         end_time = time.time()
    28         # print(f"Time elasted: {end_time - start_time:.4f} seconds")

c:\Users\yilu2\my_files\research\FELsim\experiment\SOBOL_scan.py in
    ↪compute_hessian_phi(I_dict, A_VARS, A_OBJ, delta)
   130     I_dict_plus = I_dict_plus_dict[i]
   131     I_dict_minus = I_dict_minus_dict[i]
--> 132     grad_plus_delta = compute_grad_phi(I_dict_plus, A_VARS, A_OBJ,
    ↪delta)
   133     grad_minus_delta = compute_grad_phi(I_dict_minus, A_VARS, A_OBJ,
    ↪delta)
   134     H[:, i] = (grad_plus_delta - grad_minus_delta) / (2 * delta)

c:\Users\yilu2\my_files\research\FELsim\experiment\SOBOL_scan.py in
    ↪compute_grad_phi(I_dict, A_VARS, A_OBJ, delta)
   121
   122     def compute_grad_phi(I_dict, A_VARS, A_OBJ, delta=1e-5):
--> 123         r = compute_weighted_MSE(I_dict, A_VARS, A_OBJ)[0]
   124         return 2 * compute_jacobian(I_dict, A_VARS, A_OBJ, delta)[0].T
    ↪r
   125

```

```

c:\Users\yilu2\my_files\research\FELsim\experiment\SOBOL_scan.py in
↳ wrapper(*args, **kwargs)
    24     def wrapper(*args, **kwargs):
    25         start_time = time.time()
---> 26         result = func(*args, **kwargs)
    27         end_time = time.time()
    28         # print(f"Time elasted: {end_time - start_time:.4f} seconds")

c:\Users\yilu2\my_files\research\FELsim\experiment\SOBOL_scan.py in
↳ compute_weighted_MSE(I_dict, A_VARS, A_OBJ)
    60
    61     I_list = [I_dict[var_name] for var_name in opti.variablesToOptimize]
---> 62     phi = opti._optiSpeed(I_list)
    63     num_goals = 2
    64     r_list = []

c:\Users\yilu2\my_files\research\FELsim\backend\beamOptimizer.py in
↳ _optiSpeed(self, variableVals)
    112         raise e
    113     else:
--> 114         particles = np.array(segments[i].useMatrice(particles))
    115         # Check if indice in objective dictionary
    116         if i in self.objectives:

c:\Users\yilu2\my_files\research\FELsim\backend\beamline.py in useMatrice(self,
↳ val, **kwargs)
    197         # val = val.to(torch.float64)
    198         # transformation matrix M with the size 6*6, where the new
↳ particle P'=MP
--> 199         mat = self.getSymbolicMatrice(numeric = True, **kwargs)
    200
    201         # transform into torch tensor

c:\Users\yilu2\my_files\research\FELsim\backend\beamline.py in
↳ getSymbolicMatrice(self, numeric, length, angle)
    646         phi = sp.simplify(K_simplified * g * h * (1 + sp.sin(eta) ** 2) /
↳ sp.cos(eta))
    647         Ty = sp.tan(eta - phi)
--> 648         M56 = -self.f * (1 / (self.C * self.beta * self.gamma * (self.
↳ gamma + 1)))
    649
    650

c:
↳ \Users\yilu2\programming_lang\Anaconda\envs\felisim\Lib\site-packages\torch\tensor.
↳ py in wrapped(self, *args, **kwargs)
    38     f: Callable[Concatenate[_TensorLike, _P], "Tensor"],
    39 ) -> Callable[Concatenate[_TensorLike, _P], "Tensor"]:

```

```

---> 40     @functools.wraps(f)
      41     def wrapped(self: _TensorLike, *args: _P.args, **kwargs: _P.kwargs),
      ↪-> "Tensor":
      42         try:

```

KeyboardInterrupt:

```

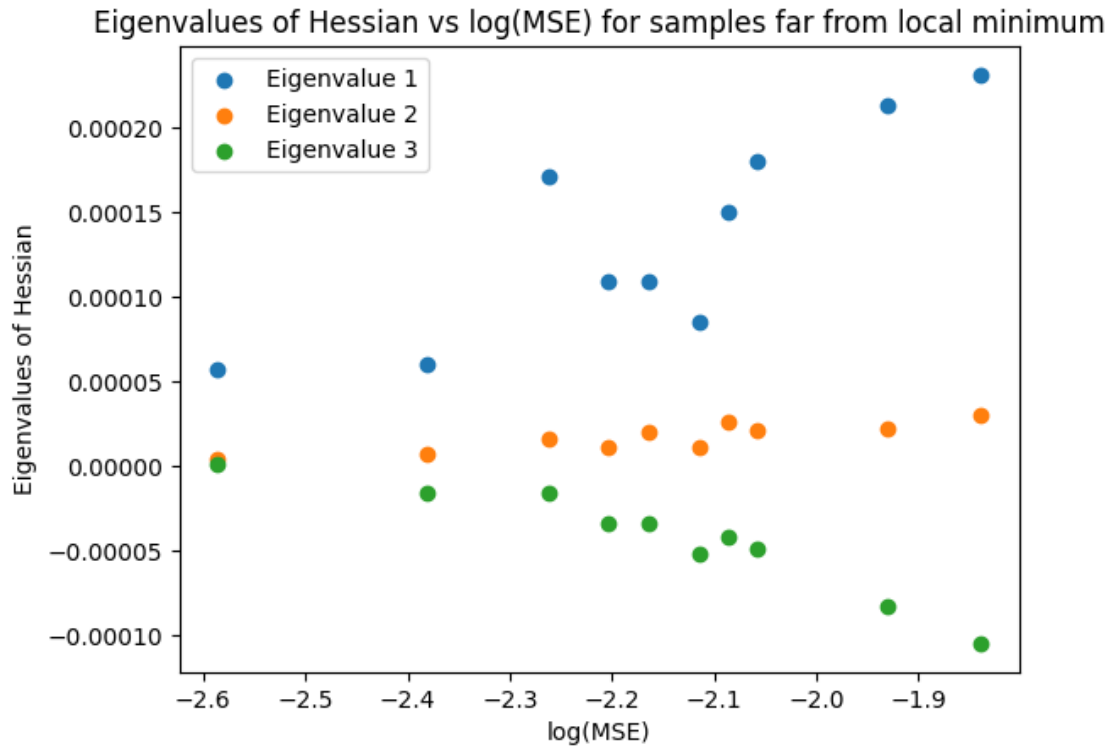
[11]: log_mse_arr=[]
      eigval_1_arr=[] # largest eigenvalue
      eigval_2_arr=[] # second largest eigenvalue
      eigval_3_arr=[] # smallest eigenvalue

      for sub_sample in res_far:
          current=sub_sample["current"]
          I_dict={"I_1": current[0], "I_3": current[1], "I_10": current[2]}

          mse=compute_weighted_MSE(I_dict, A_VARS, A_OBJ)[0]
          log_mse_arr.append(np.log10(mse))
          H=compute_hessian_phi(I_dict, A_VARS, A_OBJ)[0]
          eigvals=compute_eig(H)[0]
          eigvals.sort()
          eigvals = eigvals[::-1]
          eigval_1_arr.append(eigvals[0])
          eigval_2_arr.append(eigvals[1])
          eigval_3_arr.append(eigvals[2])

      plt.scatter(log_mse_arr, eigval_1_arr, label='Eigenvalue 1')
      plt.scatter(log_mse_arr, eigval_2_arr, label='Eigenvalue 2')
      plt.scatter(log_mse_arr, eigval_3_arr, label='Eigenvalue 3')
      plt.xlabel('log(MSE)')
      plt.ylabel('Eigenvalues of Hessian')
      plt.title('Eigenvalues of Hessian vs log(MSE) for samples far from local_
      ↪minimum')
      plt.legend()
      plt.show()

```



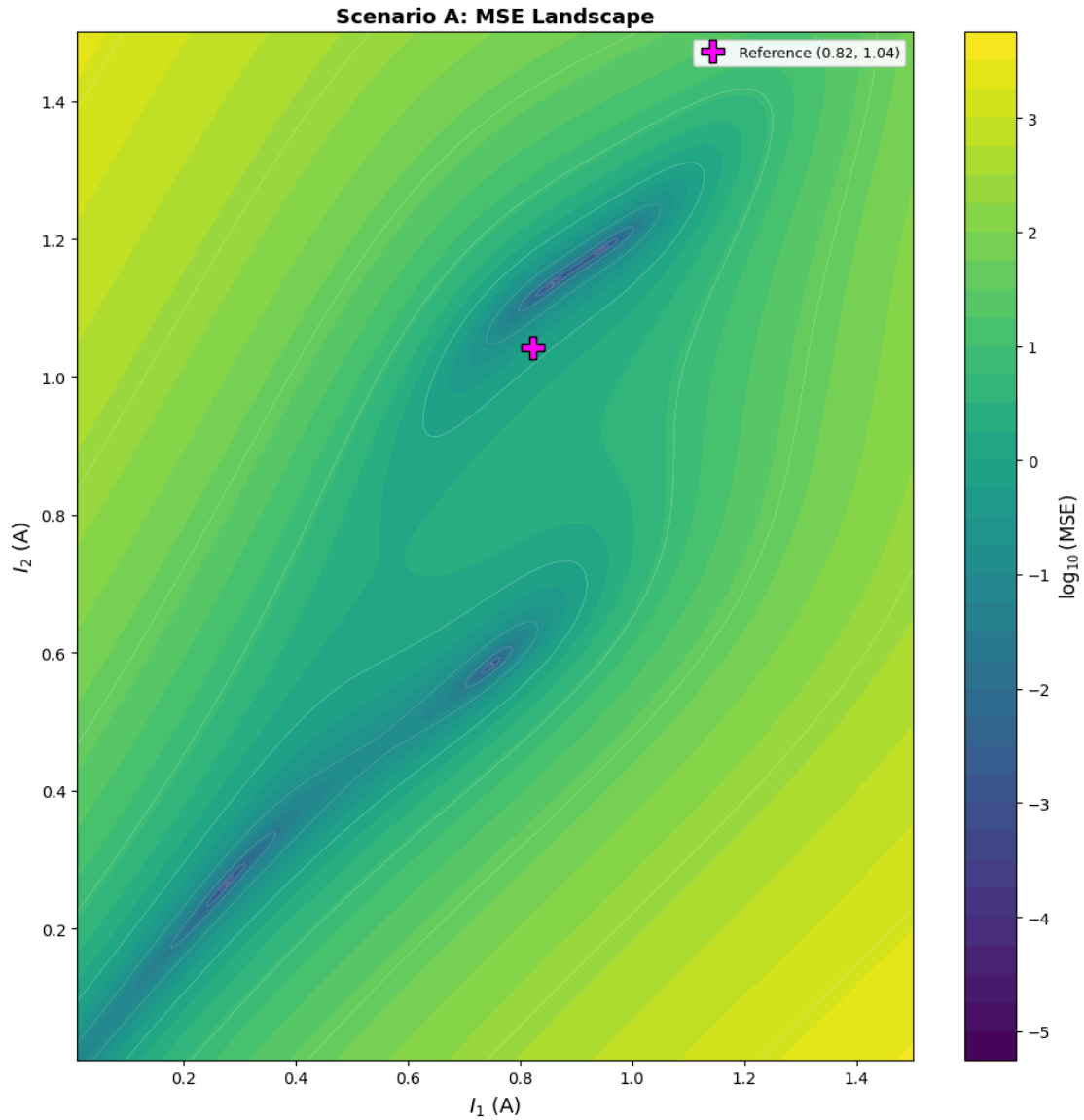
plot of eigenvector directions of samples close to the local minimum, in scenario , I_1 and I_3 as the current variables, alpha_x=0 at position 8 and alpha_y=0 at position 9

```
[2]: # produce the scan
from experiments_utils import plot_contour, load_results
with open('../results/scenario_A_landscape.pkl', 'rb') as f:
    landscape = pickle.load(f)

results_A, df_A, final_summary = load_results('../results/scenario_A_results.
    ↪pkl')

plot_contour('../results/scenario_A_landscape.pkl')
```

experiment results loaded from ../results/scenario_A_results.pkl
Grid 500 500, MSE range [9.79e-06, 3.41e+03]



```
[3]: from SOBOL_scan import get_random_samples, compute_hessian_phi, compute_eig

local_min_1 = {"I_1": 0.7512, "I_3": 0.5806}
local_min_2 = {"I_1": 0.25, "I_3": 0.27}
local_min_3 = {"I_1": 0.85, "I_3": 1.13}
local_min_4 = {"I_1": 0.95, "I_3": 1.19}

local_list = [local_min_1, local_min_2, local_min_3, local_min_4]

A_VARS = {
    1: ["I_1", "current", lambda x: x],
```

```

    3: ["I_3", "current", lambda x: x],
}

A_OBJ = {
    8: [{"measure": ["x", "alpha"], "goal": 0.0, "weight": 1.0}],
    9: [{"measure": ["y", "alpha"], "goal": 0.0, "weight": 1.0}],
}

```

```

[6]: # plot_contour('../..//results/scenario_A_landscape.pkl')
path = "../..//results/scenario_A_landscape.pkl"
with open(path, "rb") as f:
    L = pickle.load(f)
    I1_vals, I2_vals, MSE_grid = L["I1_vals"], L["I2_vals"], L["MSE_grid"]

    print(
        f"Grid {L['N_GRID']} {L['N_GRID']}, MSE range [{np.nanmin(MSE_grid):.
↪2e}, {np.nanmax(MSE_grid):.2e}]"
    )

    Z = np.log10(np.maximum(MSE_grid, 1e-16))

    fig, ax = plt.subplots(figsize=(10, 10))

    cf = ax.contourf(I1_vals, I2_vals, Z, levels=40, cmap="viridis")
    cbar = fig.colorbar(cf, ax=ax)
    cbar.set_label(r"$\log_{10}(\mathrm{MSE})$", fontsize=12)

    ax.contour(
        I1_vals, I2_vals, Z, levels=12, colors="white", linewidths=0.4, alpha=0.
↪4
    )

    ax.plot(
        0.8218,
        1.0430,
        "p",
        color="magenta",
        markersize=15,
        markeredgecolor="black",
        label="Reference (0.82, 1.04)",
        zorder=11,
    )

    ax.set_xlabel(r"$I_1$ (A)", fontsize=13)
    ax.set_ylabel(r"$I_2$ (A)", fontsize=13)
    ax.set_title("Scenario A: MSE Landscape", fontsize=13, fontweight="bold")
    ax.legend(fontsize=9, loc="upper right", framealpha=0.92)

```

```

ax.set_xlim(I1_vals[0], I1_vals[-1])
ax.set_ylim(I2_vals[0], I2_vals[-1])
fig.tight_layout()
# plt.show()

H_GN_dist=[] # compute the distance between H and H_GN
for local_min in local_list:
    bounds = np.array([0, 1.5])
    num_samples = 10
    thres_1 = 0.01
    thres_2 = 0.5
    mode = "close"
    samples = get_random_samples(local_min, bounds, num_samples, thres_1,
↪thres_2, mode)
    for sample in samples:
        J, rank_J, time_jac = compute_jacobian(sample, A_VARS, A_OBJ)
        H = compute_hessian_phi(sample, A_VARS, A_OBJ)[0]
        H_GN = compute_hessian_GN(sample, A_VARS, A_OBJ)[0]
        H_GN_dist.append(np.linalg.norm(H - H_GN, ord=1))
        eigval = compute_eig(H)[0]
        eigvec = compute_eig(H)[1]
        for val, vec in zip(eigval, eigvec):
            if val < 0:
                arrow_color = "red"
                label = "eigval<0"
            elif 0 <= val <= 0.01:
                arrow_color = "yellow"
                label = "0<=eigval<0.01"
            else:
                arrow_color = "green"
                label = "eigval>=0.01"

            plt.quiver(
                sample["I_1"],
                sample["I_3"],
                vec[0],
                vec[1],
                color=arrow_color,
                scale=15,
                label=label,
            )

handles, labels = plt.gca().get_legend_handles_labels()
by_label = dict(zip(labels, handles))
plt.legend(
    by_label.values(), by_label.keys(), fontsize=9, loc="upper right",
↪framealpha=0.92

```



```
)  
plt.show()
```

Grid 500 500, MSE range [9.79e-06, 3.41e+03]

